

TRUTHLENS
Fake News Detection System

Project Report

A Machine Learning Based Web Application for Classifying News Text as Real or Fake

Built using Python, scikit-learn, and Streamlit

Abstract

TruthLens is a web-based application that uses machine learning to help a reader judge whether a piece of news text is likely to be real or fake. The system combines a trained text-classification model with live news collection from multiple RSS feeds, so that a user can see automatic predictions on current headlines as well as check any text of their own. The core of the system is a TF-IDF text vectorizer paired with a Logistic Regression classifier, built using Python's scikit-learn library and served through an interactive Streamlit web interface. In addition to predictions, the application reports a confidence score for every result, so that a strong call and a borderline call are never presented in the same way.

This report documents the objective, design, methodology, features, and limitations of the TruthLens system, based entirely on its current, working implementation. It explains what the application does, how its machine learning pipeline works from raw text to a final prediction, how live news is collected from outside sources, and what a user sees and can do inside the application. It also honestly discusses the current limitations of the system, most importantly the small size of the sample dataset it ships with, and lists realistic directions in which the project could be extended.

Table of Contents

Abstract 2

1. Introduction..... 4

2. System Requirements 6

3. System Design and Architecture..... 7

4. Methodology 9

5. Features and Functionality..... 12

6. Live News Integration (RSS Feeds)..... 14

7. Installation and Setup 16

8. How to Use the Application 17

9. Results and Model Evaluation 18

10. Testing 19

11. Limitations 20

12. Future Enhancements 21

13. Conclusion 22

14. References..... 23

1. Introduction

1.1 Background

False and misleading information can spread quickly through news websites and social media, and it is not always easy for a reader to judge, at a glance, whether a headline or article is trustworthy. Checking every single piece of news by hand takes time, and most readers simply do not have the time or resources to verify everything they come across. This creates a real need for tools that can give a quick, automated first impression of whether a piece of text looks reliable or suspicious, so that a reader knows where to focus their own attention and their own verification efforts.

TruthLens was built to explore this idea directly: can a machine learning model, trained on examples of real and fake news text, give a useful, fast, first-pass judgment on new text it has never seen before? The project brings this idea into a working application rather than leaving it as a theoretical exercise, by connecting a trained model to both live news feeds and a simple text-entry box.

1.2 Problem Statement

The problem addressed by this project is stated as follows: given a piece of news text — a headline, a short article, or a live item pulled from a news RSS feed — can a machine learning model be trained to classify that text as REAL or FAKE with a reasonable degree of confidence, and can this capability be delivered through an interface that is simple enough for a non-technical user to use immediately, without needing to understand anything about machine learning?

1.3 Objectives

The objectives of this project are:

- To build a machine learning model that can classify news text as REAL or FAKE, based on patterns learned from labeled training examples.
- To make the model easy to retrain on a new dataset, without needing to change any code, through a simple command-line script.
- To automatically collect live news headlines from multiple public RSS feeds, so the application always has current news to display.

- To combine the trained model and the live news feed into a single, working web application that anyone can open and use.
- To present every prediction with a confidence score and a probability breakdown, rather than only a plain REAL or FAKE label, so that the strength of a given prediction is always visible.
- To make the reasoning behind the system transparent to the user, through a dedicated page that explains what the model is and how it works.

1.4 Scope of the Project

The project covers the complete pipeline from raw training data to a working, interactive web application. This includes preparing and validating training data, building and training the text-classification model, evaluating that model, automatically collecting live news from RSS feeds, and presenting all of this through a Streamlit-based user interface with several dedicated views: a live dashboard, a live-news explorer, a manual text checker, a page describing the model, and an about page describing the system as a whole.

The scope of the project does not include fact-checking claims against external knowledge bases, verifying images or video, or detecting fake news in languages other than English. The model itself is trained on English-language text; while one of the live news sources included in the application (BBC Hindi) provides Hindi-language headlines, those headlines are passed through the same English-trained model, and any prediction on non-English text should be treated with extra caution for that reason.

2. System Requirements

2.1 Hardware Requirements

TruthLens does not require any special or high-performance hardware. It runs comfortably on any standard laptop or desktop computer capable of running Python. The trained model itself is small and lightweight, since it is built on TF-IDF features and Logistic Regression rather than a large neural network, so no graphics card (GPU) is needed at any point. An active internet connection is required only for the live news feature, since fetching RSS feeds means downloading data from external news websites; the manual text-checking feature works fully offline once the model has been trained.

2.2 Software Requirements

The application is written entirely in Python and depends on a small number of well-established libraries. The exact dependencies, as declared in the project's requirements.txt file, are as follows:

Requirement	Purpose in this Project
Python 3.10 or later	The programming language the entire project is written in.
streamlit >= 1.36	Builds the interactive web application and user interface.
pandas >= 2.2	Handles tabular data — training data, live news results, and prediction tables.
scikit-learn >= 1.5	Provides the TF-IDF vectorizer and the Logistic Regression classifier.
joblib >= 1.4	Saves the trained model to disk and loads it back again.

One point worth highlighting is that the live news collection feature does not depend on any external web-scraping or RSS-parsing library. It is built entirely using two modules from Python's own standard library — urllib, to download each feed, and xml.etree.ElementTree, to read the downloaded XML content — so nothing beyond the four packages listed above needs to be installed for the whole application, live news included, to work.

3. System Design and Architecture

3.1 Overall Architecture

TruthLens is organized around two main flows that both lead into the same trained model. The first flow handles text that a user provides directly or selects from the live news list; the second flow handles the automatic collection and checking of live news headlines.

In the first flow, a user provides a piece of text, either by typing or pasting it into the "Check Text" tab, or by selecting one headline from the "Live News" tab. This text is passed to the trained model, which returns a predicted label together with a confidence score. The result is displayed to the user immediately, along with a short message encouraging verification if the prediction is FAKE.

In the second flow, the application requests the latest items from a configured set of RSS feeds. For every item retrieved, the title and summary are combined into a single piece of text and cleaned of any HTML formatting left over from the feed. Each of these combined texts is then passed through the very same trained model used in the first flow, so every live headline on the dashboard receives an automatic prediction without the user needing to click on anything.

Both flows share one trained model, stored on disk as a single file once training is complete. The model only needs to be trained once — or re-trained deliberately, if the training data changes — after which the saved file is simply loaded every time the application starts. This keeps the application fast to open, since it does not need to retrain a model from scratch on every run.

3.2 Module Description

The project's code is organized into a small number of clearly separated files, each responsible for one part of the system. This separation keeps the user-interface code and the machine-learning code independent of each other, so that either one can be changed without needing to touch the other.

- `app.py` — The Streamlit application itself. It defines the layout of the web page (sidebar, tabs, and the separate "Model Info" and "About" pages), handles user interaction such as button clicks, text entry, and source selection, and calls into the other modules whenever it needs a prediction or a batch of live news. It does not contain any machine-learning logic of its own; it only displays results calculated elsewhere.

- `src/detector.py` — This module contains all of the machine-learning logic in the project: preparing and validating a training dataset, building the TF-IDF plus Logistic Regression pipeline, training the model, saving and loading it from disk, and generating a prediction (with a confidence score) for one piece of text at a time.
- `src/news_scraper.py` — This module is responsible for fetching and parsing RSS feeds from the configured news sources. It downloads each feed, parses the XML content, extracts the title, summary, publication date, and link for every item, removes duplicate entries, and hands the results back as a table (a pandas DataFrame) that the rest of the application can work with directly.
- `train_model.py` — A short command-line script that allows the model to be retrained without opening the web application at all. It reads a CSV file, trains a model on it using the logic defined in `detector.py`, and saves the resulting model to disk.
- `scrape_news.py` — A short command-line script that allows live news to be collected without opening the web application. This is useful for quickly checking what the current RSS feeds contain, or for saving a fresh batch of news items to a CSV file, for example as a starting point for building a larger, more realistic training dataset later on.
- `data/news_sample.csv` — A small, ready-to-use dataset of example REAL and FAKE news text. This file is what the model is trained on if no other dataset is supplied.
- `models/fake_news_model.joblib` and `models/metrics.json` — The saved, trained model file itself, and a small companion file that records how well that particular model performed when it was trained (its accuracy, and how many rows it was trained and tested on).

3.3 Data Flow Summary

At a high level, data moves through the system in the following order. First, during training: a labeled CSV file is read, cleaned, and split into a training portion and a validation portion; a TF-IDF vectorizer learns which words and word-pairs matter from the training text; a Logistic Regression classifier learns to associate those word patterns with the REAL or FAKE label; the trained pipeline is saved to disk as a single file, along with a small record of its accuracy.

Second, during ordinary use of the application: the saved model is loaded once when the application starts; any text a user types in, or any live headline collected from an RSS feed, is passed through the same loaded model; the model converts that text into TF-IDF features and

returns a probability for each class (FAKE and REAL); the higher of the two probabilities becomes the predicted label, and is shown to the user as the confidence score.

4. Methodology

4.1 Data Preparation

Before any training happens, the project's data-handling code checks that a given CSV file is usable and puts it into a consistent shape. Rather than requiring an exact, fixed column layout, the system looks for a text column under any of several common names — text, title, headline, article, content, or news — and a label column under any of several common names — label, target, class, or category. This makes it straightforward to plug in a differently-formatted dataset later without needing to rename columns by hand.

Label values are also normalized rather than required to match one exact spelling. Any of the following values are recognized and converted into a plain FAKE or REAL label internally: fake, false, 0, misleading, rumor, and rumour map to FAKE; real, true, 1, reliable, and genuine map to REAL. This matching is not case-sensitive. Blank rows, and rows with unusable text, are automatically dropped before training begins, and a clear error is raised if a file with the wrong structure altogether — for example, missing both a recognizable text column and a recognizable label column — is provided.

4.2 Text Vectorization: TF-IDF

Machine learning models work with numbers, not raw words, so the first real step in the pipeline is converting news text into a numerical form the model can use. TruthLens does this using a technique called TF-IDF, short for "Term Frequency, Inverse Document Frequency." In plain terms, TF-IDF looks at how often a word appears in one particular piece of text (its term frequency), and weighs that against how common the word is across all the text the model has seen (its inverse document frequency). Words that appear often in one document but rarely elsewhere — words that are unusually distinctive to that piece of text — end up with a higher weight than extremely common words that appear almost everywhere and therefore carry little useful information for telling documents apart.

In this project, the TF-IDF step is configured to look at both single words and pairs of consecutive words (what is technically called a 1-2 n-gram range), so that short, meaningful phrases are captured and not just isolated words in isolation. Common English "stop words" — words like "the," "and," or "is," which appear constantly and rarely help distinguish one piece of text from another — are removed automatically. All text is also converted to lowercase before comparison, so that capitalization differences do not affect the result.

4.3 Classification Model: Logistic Regression

Once a piece of text has been converted into TF-IDF numbers, TruthLens uses a Logistic Regression model to make the actual REAL-versus-FAKE decision. Logistic Regression is a well-established and widely-used classification technique that estimates the probability of an outcome — in this case, the probability that a given piece of text belongs to the FAKE class versus the REAL class — based on the weighted combination of its input features. It is a common and reliable first choice for text-classification tasks of this kind, particularly when the dataset is not extremely large, because it trains quickly, is straightforward to interpret, and tends to generalize reasonably well without requiring heavy computing resources.

The specific Logistic Regression settings used in this project are worth noting. The model is configured with `class_weight` set to "balanced," which means that if one label (FAKE or REAL) happens to be more common than the other in the training data, the model automatically adjusts so that it does not simply favor the majority class by default. It uses the "liblinear" solver, which is a solid, efficient choice for smaller text-classification datasets, and allows up to 1000 optimization iterations to make sure the model has a chance to properly converge on a good solution during training.

4.4 Model Training and Validation

When a new model is trained, the available labeled data is split into a training portion and a validation (test) portion, so that the model's accuracy can be measured on examples it did not directly learn from. In this project, that split sets aside 25 percent of the data for validation, and the split is stratified, meaning the proportion of FAKE and REAL examples is kept consistent between the training portion and the validation portion, rather than being split purely at random.

This split-and-validate approach is only used when there is enough data to make it meaningful — specifically, when there are at least 10 total rows and at least 2 examples of the smaller class. If a dataset is smaller than that, the system instead trains on all of the available data without holding any of it back, since with only a handful of examples, setting some aside for testing would leave almost nothing to train on and would not produce a trustworthy accuracy figure either way. In that fallback case, the reported accuracy is simply recorded as complete (1.0), since the model is being evaluated on data it has already seen — this is a reasonable practical fallback for very small demonstration datasets, but it is not a genuine held-out accuracy measurement, and should not be read as one.

4.5 Making a Single Prediction

Once a model has been trained and saved, using it to check a new piece of text is a quick, three-step process. The input text is transformed into TF-IDF features using the exact same vectorizer settings that were used during training; the Logistic Regression model then outputs a probability for each class, FAKE and REAL; and finally, whichever class has the higher probability becomes the predicted label, with that probability reported back as the confidence score. Both the FAKE probability and the REAL probability are also kept and shown separately, rather than only showing the winning one, so a user can see, for example, that a prediction of REAL at 52 percent confidence is a very different situation from a prediction of REAL at 97 percent confidence, even though both would display the same label.

5. Features and Functionality

The TruthLens interface is organized into a sidebar for navigation and settings, plus a set of pages and tabs for the main content. This section walks through every part of the interface and what it does.

5.1 Sidebar

The sidebar is visible at all times and contains the TruthLens branding, navigation buttons for each page (Dashboard, Live News, Check News, Model Info, and About), a list of news sources with a checkbox next to each one (all selected by default), a slider that controls how many items to fetch per news feed (with options of 3, 5, 8, 10, 15, or 20), and a "Refresh News" button that clears any cached live news and fetches a fresh batch. Below that, the sidebar also displays live information about the current model: the total number of rows it was trained on, its recorded accuracy, and whether it is running from a previously saved model file or was just trained fresh from the sample data.

5.2 Dashboard

The Dashboard tab is the main overview screen. It automatically fetches the latest headlines from every selected news source, runs every single one through the trained model, and then shows three summary numbers at a glance: how many headlines were checked in total, how many were marked FAKE, and how many were marked REAL. Below these numbers, a full table lists every checked headline along with its source, its prediction, its confidence, and its individual FAKE and REAL probabilities. A second table beneath that specifically isolates and lists only the headlines the model marked as FAKE, so that the items most worth a closer look are easy to find without scrolling through the full list.

5.3 Live News

The Live News tab also fetches the latest headlines from the selected sources, but is built for exploring and checking one item at a time rather than seeing everything at once. It displays a table of all fetched headlines (source, feed, title, publish date, and a clickable link to the original article), and lets the user pick any one of them from a dropdown list. Once a headline is selected, its title, summary, and a link to the original article are shown, along with the underlying text in an editable text box.

A deliberate safeguard is built into this tab: if the user edits the text in that box before clicking "Check selected headline," the application detects the change and shows an error message

asking the user to either refresh the text or use the original RSS wording, rather than silently checking modified text as if it were the genuine live headline. This keeps the "live news check" feature honest — a prediction shown under this tab can always be trusted to reflect the actual, unmodified text that was pulled from the news source.

5.4 Check Text

The Check Text tab is the simplest and most direct way to use the model: a large text box where any user can paste or type a headline or short piece of article text, and a "Check news" button that immediately runs it through the model. The result — predicted label, confidence percentage, a visual progress bar, and the individual FAKE/REAL probability split — is displayed right below the button. If the text box is left empty, the application shows a gentle warning instead of attempting a prediction on nothing.

5.5 Model Info Page

This page exists purely to explain the system to the user in plain terms. It states clearly that the underlying model is a Logistic Regression classifier working on TF-IDF-processed text, explains what REAL and FAKE mean as outputs, lists the kinds of evaluation metrics relevant to a model like this (accuracy, precision, recall, and F1-score), and includes a direct reminder that a prediction from this system is based on learned patterns, not verified fact, and should be checked against reliable news sources.

5.6 About Page

This page introduces TruthLens as a whole and walks through its process in four simple steps: the user enters news text; that text is processed into numerical features; the trained model generates a prediction from those features; and the system displays the result together with confidence information. It closes with the same honest caveat found throughout the application — that no automated system can guarantee whether any given article is true or false, and that important information should always be verified using multiple, reliable sources.

6. Live News Integration (RSS Feeds)

6.1 News Sources Used

The live news feature currently draws from five news outlets, across thirteen individual RSS feeds in total, as summarized below.

Source	Feeds Included
Times of India	Top Stories, Most Recent
Indian Express	Latest, India
NDTV	Top Stories, India, World
BBC	Top Stories, World, Technology, Business, Asia
BBC Hindi	Latest

Every one of these sources can be individually switched on or off from the sidebar, so a user can, for example, look only at BBC feeds, or only at Indian sources, rather than always seeing all five at once.

6.2 How a Feed Is Fetched and Read

For every selected feed, the application sends a standard web request to the feed's URL, identifying itself with a normal browser-style header so that the request is treated the same way a browser's request would be. The downloaded content is then parsed as XML. Because RSS and Atom (a related but slightly different feed format) structure their content differently, the parser first looks for standard RSS "item" entries, and falls back to Atom-style "entry" tags if none are found, so that a feed published in either format is handled correctly.

From each entry, the application extracts the headline title, a short summary or description, the publication date, and a link back to the original article. Any HTML formatting left inside the summary text (bold tags, links, and similar markup that news feeds commonly include) is stripped out, leaving clean, readable plain text. The title and summary are then joined together into a single block of text, which is exactly what gets passed into the trained model for a prediction.

6.3 Error Handling

News websites and their RSS feeds are outside of this project's control, and any one of them can occasionally be slow, temporarily unavailable, or return unexpected content. To keep the application usable even when that happens, every individual feed is fetched inside its own error-handling block. If one particular feed fails — whether due to a network issue, a timeout,

or unexpected feed content — that single failure is recorded and shown to the user as a clearly labeled error message, while every other feed continues to be fetched normally. A single failing source is therefore never able to bring down the entire dashboard.

6.4 Removing Duplicate Entries

Because some outlets publish overlapping stories across more than one of their own feeds (for example, a top story appearing in both a "Top Stories" feed and a general "Latest" feed from the same outlet), the application removes duplicate entries after all feeds have been collected. This is done in two passes: first by removing repeated article links, and then by removing repeated title-and-source combinations. This keeps the dashboard and live news list free of the same headline showing up more than once.

7. Installation and Setup

7.1 Prerequisites

Before setting up the project, the following should already be installed on the system:

- Python, version 3.10 or newer.
- pip, the Python package manager (this is included automatically with most Python installations).

7.2 Step-by-Step Setup

Step 1 — Obtain the project files. Download or clone the project folder onto the local machine, and open a terminal or command prompt inside it.

Step 2 — Install the required Python packages. From inside the project folder, run:

```
pip install -r requirements.txt
```

This single command installs Streamlit, pandas, scikit-learn, and joblib — the only four external packages the project depends on.

Step 3 — Train the model. Before the application can make predictions, a trained model file needs to exist. This is created by running:

```
python train_model.py
```

This trains a model on the included sample dataset and saves it inside the models folder, along with a metrics file recording its accuracy. This step only needs to be repeated later if the training data changes.

No API keys, external accounts, paid services, or database setup are required at any point — every part of the setup process happens locally on the user's own machine.

8. How to Use the Application

8.1 Starting the Application

Once the setup steps in the previous section are complete, the application itself is started with:

```
python -m streamlit run app.py
```

Running this command starts a local web server and prints a local address (typically something like `http://localhost:8501`) in the terminal. Opening that address in a web browser loads the TruthLens interface.

8.2 Checking the Live Dashboard

On opening the application, the Dashboard page is shown by default. Selecting or de-selecting sources in the sidebar, or adjusting the "items per source" slider, and then pressing "Refresh News," updates which headlines are fetched and displayed. The dashboard immediately shows how many of the fetched headlines were marked FAKE versus REAL, along with the full results table.

8.3 Checking One Live Headline

Switching to the Live News tab shows the same kind of fetched headlines, but allows picking one specific headline from a dropdown list to look at in detail — its full summary, a link to read the original article, and a "Check selected headline" button to run that specific item through the model.

8.4 Checking Custom Text

Switching to the Check Text tab allows any text at all — not limited to the configured news sources — to be pasted or typed into the text box, and checked with the "Check news" button.

8.5 Using the Command-Line Tools Directly

Both the model training and the live news fetching can also be run directly from a terminal, without opening the Streamlit application at all — useful for quick checks, automation, or for building a larger dataset over time.

To fetch the latest headlines and print them in the terminal:

```
python scrape_news.py --limit-per-feed 20
```

To save the same fetched headlines into a CSV file instead:

```
python scrape_news.py --limit-per-feed 20 --output  
data/latest_news.csv
```

To retrain the model on a different dataset:

```
python train_model.py --data path/to/your_dataset.csv
```

9. Results and Model Evaluation

9.1 *Evaluation Metrics Recorded*

Every time the model is trained, a small metrics file is saved alongside it, recording how the model performed. For the model currently shipped with the project, trained on the included sample dataset, the recorded figures are as follows:

Metric	Value
Accuracy	80% (0.8)
Total rows in dataset	40
Rows used for training	30
Rows held out for validation	10
FAKE examples	20
REAL examples	20

9.2 *Reading These Numbers Honestly*

An accuracy of 80 percent means that, of the 10 examples set aside for validation during training, the model correctly predicted the label for 8 of them. It is important to be direct about what this number does and does not represent. A validation set of only 10 rows is extremely small — a single incorrect prediction on that set changes the accuracy figure by a full 10 percentage points. This figure is useful as a demonstration that the training pipeline works correctly end-to-end, and that the model is capable of learning a real pattern from labeled examples, but it should not be read as a dependable estimate of how the model would perform on real-world news at large. A trustworthy accuracy estimate would require a much larger and more varied validation set.

The included sample dataset itself is also intentionally small and fairly clear-cut: its REAL examples are written in a formal, institutional, news-reporting style, and its FAKE examples describe obviously exaggerated or implausible claims. This makes the dataset well suited for demonstrating that the whole pipeline — from raw text to a trained, working model — functions correctly, but it does not represent the more subtle, carefully-worded misinformation that is often the hardest to catch in the real world.

10. Testing

As a Streamlit application built around a single trained model, TruthLens is best verified through direct, functional use of each feature, rather than through a separate automated test suite. The following checks describe how each part of the system can be exercised and confirmed to behave as designed.

10.1 Model and Prediction Checks

- Entering a piece of text that closely resembles the wording style of the sample dataset's REAL examples (formal, institutional, source-attributed language) and confirming it is classified as REAL.
- Entering a piece of text that closely resembles the sample dataset's FAKE examples (exaggerated, unverifiable, urgent-sounding claims) and confirming it is classified as FAKE.
- Checking that every prediction, on both the Dashboard and the Check Text tab, always displays a confidence percentage and a FAKE/REAL probability split alongside the label, rather than the label alone.

10.2 Live News Checks

- Confirming that toggling a news source off in the sidebar removes that source's headlines from both the Dashboard and Live News tabs after a refresh.
- Confirming that changing the "items per source" slider changes the number of headlines fetched per source accordingly.
- Selecting a live headline in the Live News tab, deliberately editing the text box, and confirming that the application correctly blocks the check with an explanatory error, rather than silently evaluating the edited text.
- Confirming that if a single RSS feed is unreachable or returns an error, the rest of the dashboard still loads normally, with the failing feed listed separately under "Feed errors."

10.3 General Application Checks

- Confirming that all five navigation pages (Dashboard, Live News, Check News, Model Info, About) load correctly from the sidebar.
- Confirming that the sidebar's reported "Training rows," "Accuracy," and "Model" values match what is stored in models/metrics.json.

- Confirming that submitting an empty text box on the Check Text tab shows a warning instead of attempting a prediction.

11. Limitations

Being transparent about what this project is, and is not, is an important part of documenting it responsibly. The following limitations are worth keeping in mind by anyone using or building on TruthLens.

- Small training dataset. The model included with the project is trained on only 40 labeled examples. This is enough to demonstrate the full pipeline working correctly, but far too small to produce a model that reliably generalizes to the huge variety of real-world news text.
- Clear-cut example style. The sample dataset's REAL and FAKE examples are written in noticeably different styles from one another, which makes them easier for a model to separate than genuinely subtle, well-written misinformation would be.
- A prediction is a statistical estimate, not a fact-check. The model looks only for word and phrase patterns it has seen before; it does not verify any claim against real-world sources, databases, or events. A confident-looking prediction is still just a pattern match, not proof.
- English-language model. The trained model is built on English text. The BBC Hindi feed is included as a live news source, but its headlines are still evaluated by the same English-trained model, and predictions on non-English text should be treated with additional caution.
- Dependence on external RSS feeds. The live news feature depends on outside news websites keeping their RSS feeds online and correctly formatted. While the application is built to keep working if one feed fails, it has no control over the availability or accuracy of the source feeds themselves.

12. Future Enhancements

The current version of TruthLens is a complete, working system, but there are several realistic directions in which it could be extended and improved.

- Training on a larger, more realistic dataset. Replacing or expanding the current 40-row sample dataset with a much larger collection of real-world labeled news articles would be the single most impactful improvement, and would give the accuracy figures reported by the system genuine, trustworthy meaning.
- Trying stronger modeling techniques. Once a larger dataset is available, it would be worth comparing the current TF-IDF plus Logistic Regression approach against other techniques, such as gradient-boosted models or a fine-tuned transformer-based language model, to see whether they offer a meaningful accuracy improvement that justifies their added complexity.
- Adding source-credibility signals. Alongside the text-only prediction, incorporating information about the publisher or domain (such as a known reputation or publishing history) could give the model additional, complementary signals beyond the wording of the article itself.
- Explaining individual predictions. Showing which specific words or phrases in a piece of text pushed the model's decision toward FAKE or toward REAL would make each prediction easier to understand and to trust, rather than presenting the model as a black box.
- Expanding language and source coverage. Adding more news sources, and training separate models for languages beyond English, would make the live news dashboard more broadly useful, particularly for the non-English feeds already included as sources.

13. Conclusion

TruthLens successfully demonstrates a complete, working pipeline for automated news-text classification, starting from raw labeled text and ending in a live, interactive web application. The system combines a straightforward and well-understood machine learning approach — TF-IDF text vectorization paired with a Logistic Regression classifier — with a practical, real-world data source in the form of live RSS news feeds, and presents both through a clean, accessible Streamlit interface.

Beyond the core prediction feature, the project pays close attention to the details that make a system trustworthy and usable in practice: predictions are always shown with a confidence score rather than a bare label, the live-news-check feature is protected against being quietly bypassed with edited text, individual feed failures are isolated so they cannot break the whole dashboard, and the application is honest with its users about the fact that a machine learning prediction is not the same thing as a verified fact.

The most significant current limitation is the small size of the sample dataset the model is trained on, and this is also the clearest next step: with a larger and more realistic set of labeled examples, the same pipeline documented in this report would be able to produce a genuinely more reliable and broadly useful fake-news detector, without needing any change to the overall system design.

14. References

The following official documentation resources correspond to the core technologies used in this project.

- Python Software Foundation — Python 3 Documentation. <https://docs.python.org/3/>
- scikit-learn Developers — scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>
- Streamlit Inc. — Streamlit Documentation. <https://docs.streamlit.io/>
- pandas Development Team — pandas Documentation. <https://pandas.pydata.org/docs/>
- joblib Developers — joblib Documentation. <https://joblib.readthedocs.io/>